01 HANDWRITTEN MNIST

ASSIGNMENT 1 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import models
from tensorflow.keras import layers

# LOADING THE DATASET
train_data = pd.read_csv("HandWritten MNIST/mnist_train.csv")
test_data = pd.read_csv("HandWritten MNIST/mnist_test.csv")

print(train_data.shape)
train_data.head()

# TRAIN TEST SPLIT
X_train = train_data.drop("label", axis = 1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis = 1).values
y_test = test_data["label"].values

print("Training data: ", X_train.shape)
print("Testing data: ", X_test.shape)

# NORMALIZE THE PIXEL VALUES
X_train = X_train.astype("float32")/255.0
X_test = X_test.astype("float32")/255.0

# DEFINING MODEL ARCHITECTURE
model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.summary()

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer = 'adam',
    loss = "sparse_categorical_crossentropy",
    metrics = ['accuracy']
)

history = model.fit(
    X_train,
    y_train,
    epochs = 11,
    batch_size = 32,
    validation_split = 0.2
)

# EVALUATE THE MODEL
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# PLOT TRAINING LOSS AND ACCURACY
plt.plot(history.history['loss'], label="Training loss")
```

```
plt.plot(history.history['val_loss'], label="validation loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation loss")
plt.show()
```

```
# PREDICT THE OUTPUT
prediction = model.predict(X_test)

y_pred = np.argmax(prediction, axis = 1)
y_true = y_test

print("Predicted: ", y_pred[:10])
print("Actual: ", y_true[:10])
```

02 FASHION MNIST

ASSIGNMENT 1 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import SGD

train_data = pd.read_csv("Fashion MNIST/fashion-mnist_train.csv")
test_data = pd.read_csv("Fashion MNIST/fashion-mnist_test.csv")

# TRAIN TEST SPLIT
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

print("Training data: ", X_train.shape)
print("Testing data: ", X_test.shape)

# NORMALIZE THE PIXEL VALUES
X_train = X_train / 255.0
X_test = X_test / 255.0

# DEFINING MODEL ARCHITECTURE
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()

# COMPILING AND TRAINING THE MODEL
model.compile(
    optimizer=SGD(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs = 10,
    batch_size=32,
    validation_split=0.1
)

# EVALUATE THE MODEL
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# PLOT TRAINING LOSS AND ACCURACY
plt.plot(history.history['loss'], label='Training loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.title('Training and Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
```

```
plt.legend()
plt.show()
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
# PREDICT THE OUTPUT
prediction = model.predict(X_test)

# Get predicted class
y_pred = np.argmax(prediction, axis=1)

# Get actual class
y_true = y_test

# Print first image prediction
print("Predicted:", y_pred[0])
print("Actual:", y_true[0])
```

03 SIGN LANGUAGE MNIST

ASSIGNMENT 1 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import SGD

# LOADING DATASET
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")

# TRAIN TEST SPLIT
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

encoder = LabelEncoder()

y_train = encoder.fit_transform(train_data["label"])
y_test = encoder.transform(test_data["label"])

print("Training data: ", X_train.shape)
print("Testing data: ", X_test.shape)

# NORMALIZE PIXEL VALUES
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# DEFINING MODEL ARCHITECTURE
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation = "relu"),
    Dense(64, activation = "relu"),
    Dense(24, activation = "softmax")
])

model.summary()

# COMPILE AND TRAIN MODEL
model.compile(
    optimizer = SGD(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs = 10,
    batch_size = 32,
    validation_split=0.2
)

# EVALUATE THE MODEL
test_loss, test_accuracy = model.evaluate(
    X_test, y_test)
```

```
print("Test Loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# PLOT
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

04 CIFAR-10

ASSIGNMENT 1 • CODE REVISION SHEET

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam

# LOADING DATASET
data = pd.read_csv("CIFAR10/cifar-10.csv", dtype="uint8")

print(data.head())
print(data.shape)

# TRAIN TEST SPLIT
X = data.drop("label", axis=1).values
y = data["label"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training: ", X_train.shape)
print("Testing: ", X_test.shape)

# NORMALIZE PIXEL VALUES
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

model = Sequential([
    Input(shape=(3072,)),
    Dense(512, activation="relu"),
    Dense(256, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.2
)

# EVALUTE THE MODEL
test_loss , test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# PLOT
```

```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.show()
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
# PREDICT THE OUTPUT
prediction = model.predict(X_test)

y_pred = np.argmax(prediction, axis=1)
y_true = y_test

print("Predicted: ", y_pred[:10])
print("Actual: ", y_true[:10])
```